# CSC384

Intro AI

Last updated February 2, 2026

# Contents

# 1 Course Introduction

**Note: this course is not about machine learning. Consider CSC311 instead.**

What should I do, going forward? It's very hard to predict where it's going to go. AI is a field in CS was a field that was very lucrative, which prompted a lot of people to go into it. It's still a lucrative field for those with specific knowledge (like high level AI models), but the entry level, the things that interns used to do, is being replicated by LLMs.

It's time to view computer science not as a subject in its own right, but a way as we view mathematics. Lots of people see mathematics to be applied (unlike the purists). We need to see computer science as a *tool, a skillset* that can be applied to lots of different areas. CS can be applied to geology, healthcare, or robotics. You need to be able to apply CS into specific areas, and that can help a lot of people.

People, in addition to CS, they have an adjacent thing they are applying the CS to. You don't have to do that, but that might help you.

About the loss of creativity: do the AI(s) feel creative too?

The loss of creativity is a problem that has been looked at for a long time. Likewise how photography didn't replace paintings and instead made paintings more valuable, people might not want to buy AI-generated movies. Or it's hard to tell, and everything written here might as well be invalid.

Does anyone say "please" to ChatGPT? Do you need to? Do you feel bad for being mean to the model? Do you want to experiment and see what happens when you are mean to the model? (Unless there are TOS on the platform you are using it. If it's self-hosted…)

## 1.1 What's Taught Here?

Intelligence is a hard thing to quantify. There are many measures of intelligence (IQ or GPA). However, none of these are perfect measures of intelligence. One working definition we can use is: **doing the most optimal thing based on some reward function.** Whenever you do an action, you get some reward for it. If you are a biological agent, you will maximize the reward if you are intelligent.

Then, there comes this notion of *artificially intelligence.* We want to create intelligence, rather than existing in nature.

Let's consider a pre-historic caveman. It's semi-intelligent. Can they survive in the world that they exist in? The longer this caveman survives, the more reward it can accumulate, and the more intelligent it is. For this caveman to survive, there are lots of tasks to do.

How does a caveman determine an apple is edible or not? What about a situation where the caveman will die if they don't eat it?

Options?

- Look for something else
- Just try it
- Have someone else try it (if that's possible)

*The apple presented in the slides ended up being poisonous, and touching it is poisonous.* Information can fool you, and you will have to make judgement calls. Now that you've done that, everyone else knows. Data has been collected.

Why was this asked? There are different ways to learn. Learning by experience or learning from another source.

How do you know that an apple is rotten if you open it apart? Maybe part of your biology can know that something is wrong? That something is inherent that you have within you, that tells you that the apple isn't good? If you smell something bad?

If you've never seen a rotten apple, that's okay, but you have seen fresh apples. If it matches it most ways, but is slightly different, then could it be poisonous?

Would a newborn have certain things that it will avoid? Maybe the baby doesn't know that they are avoiding? We have the grip response inherently.

## 1.2 The Pyramid of AI

This is not an objective thing. These are loose terms that people will use slightly differently.

Artificial intelligence is built using machine learning, which can occur through:

- Statistical learning (from data)
- Imitation learning (from demonstration)
- Evolutionary learning (from evolution)
  - You are not learning as an individual. There is nothing in a biology that taught someone the grip response. But the population as a whole has learnt. Babies aren't born with a completely blank slate in their mind.
- Reinforcement learning (from experience)

Of course, this is just how this lecturer said it to be. ML is an umbrella term that captures a ton of things.

But what is this course about? It's not about one specific type of learning. It's about learning in general. This course covers a whole bunch of different things so you can choose to study more types of AI.

Almost all the learning done in practice is statistical learning, but RL is done in research and specific industries like autonomous vehicles.

- Statistical learning requires data being collected, but no consequence to you when you learn it. What happens when you don't have the data?
- Reinforcement learning is when you learn from experience. You don't need data, but experimenting on yourself has risks. In other words, **it has consequence.** You pay to reinforcement learn, sometimes without any second chance.

Is one better/more important than the other?

For many problems, acquiring data is very hard. We have the models, we have the algorithms, but where's the data? That's why companies are obsessed with data. The more data we have, the more we can learn, the more we can train, and the more money we can extract. There are many problems where, if you wanted to get the data, you couldn't. Either too risky, too dangerous, or too hard.

No one would want to volunteer for a first try. Hence we start by mocking – having learning be done in a safer environment. But it's hard. If we wanted to test out autonomous vehicles, would it be trusted?

Reinforcement is sometimes a necessity if we don't have data.

We can combine all the data together. Use reinforcement learning to do statistical learning, and the other way around.

## 1.3 The History With AI

Historically, AI started in the 1950s. AI progress is exponential, not linear.

The Dartmouth conference is where a group of engineers and scholars got together. They tried to figure out a way to create tools that. Can simulate or talk to humans

realistically. Could intelligence be simulated? When the simulations can talk, this is the first time the term “artificial intelligence” has been coined.

This started off a hype train of research. They spent the whole summer brainstorming ideas, and nothing came out of it.

It wasn’t until 10 years later, where the first semblance of AI came out, known as the ELIZA chatbot. You can find mimics of it today (it’s really bad). The way it works is purely through string manipulation. For example:

- It asks you a question (“are you feeling sad today?”)
- Why are you feeling sad today?
- It seems like you’re sad because [YOUR REASON].
- I see.

You can try around with it but it won’t do anything more than that. But at that time, people were convinced that they were talking to someone human-like. There is hype, there was fear, and there was a mix of everything. But in today’s standards, this seems like nothing.

And then after that, the first AI winter hit. People were promised a ton of things with AI, and then none of that really happened. There was this dark period that lasted for a while.

After symbolic learning phased off, export learning arrived. If symbolic learning was a bunch of IF statements, export learning transformed those IF statements into better IF statements. Wouldn’t that solve the problem? If you wanted to design an AI that could play chess, maybe ask a chess player. Re-write what they do in code. This sounds like the most obvious thing today, but that was revolutionary at that time. Encode what people do and encode it in programs. Deep blue (chess bot) was built with expert learning. It’s conditional logic that mimic what the experts did.

In the early 2000s, statistical learning became popular. SVMs started taking off in 2001. Why did this happen? This theory existed for quite a bit before 2001. These had to happen:

- The theory got sound

- We got the hardware that could process the algorithm
- We have a lot of data to run the hardware of

In 2006, Netflix issued a $1M prize that could create an algorithm that could help rank movies. This is back when Netflix sent DVDs in the mail. They want to intelligently rank movies given user data.

In 2009, the ImageNet dataset was released. Now, who can create the best image recognition model? But the accuracy was very low.

Here comes Deep learning. `AlexNet` came with an absurdly high accuracy. And then today comes LLMs.

All of this happened in the span of 70 years. This sounds like a long time but isn't. What happened between 2022 and 2025? AI in 2022 was not the same as AI in 2025.

Where are we going to go forward? Reasoning models, hybrid learning, or what? These are predictions. (Reasoning models are models that prompt itself back and think through a series of answers to reach to an answer)

Why do we want reasoning? This helps us understand how the model works and allows us to check their answers. For us to trust what AI is saying, maybe we need to have it to show us the logic and prove things **using a sound logical argument that lets us know that what the AI is saying is correct.** Without any reasoning models, no one is going to apply AI into healthcare. Or people will do it anyway.

Today, we are still mimicking chain of thought. Just "show us your logic." Just generate more text that sounds logical. It's like writing a proof that you yourself can't verify. Unfortunately that's not going to work.

This is where symbolic learning has to resurge. Symbolic learning is good at logic. It will never mess up with logic, and it will never make an error since logic can be proved.

How do we make a system that **guarantees** that every step in the chain of thought will logically follow from the previous step? That is why this course involves a section of knowledge representation and reasoning. **How do we teach computers how to do proofs?**

# 2 Search And Heuristics

Search processes are a way to intelligently make decisions. There is going to be a lot of assumptions.

In a typical RL problem, we have an environment. A place we are making decision. I am an individual, and "agent", who is making decisions within an environment, in a fixed set of states.

Clearly, the word is continuous. Or is it? Assuming things are discrete are not as bad, since if it works on a computer, we're going to have to make it discrete anyways. So, this is a question is, what is the granularity of the discretization? Of course, this requires more computation. Using math, it's possible to make this continuous, but let's just keep it discrete.

Search processes is a framework for intelligent decision making. **Notation may be different across sections. Be comfortable with different notations. Hence, state what your variables are for a problem.**

Given:

- A discrete set of states $S$
  - The possible states of the environment
- An initial state $s_0 \in S$
  - The initial state the environment will start in
- A discrete set of actions $A(s)$
  - A set of actions that the agent can take from each state
  - Assumes the set of actions I can take is **discrete.**
  - *What action will I take?*
- A **deterministic** policy, $\alpha : S \rightarrow A$
  - Defines which action the agent will take given the state
  - *How am I going to choose the action?*

  - Not random and will always be the same

- A transition function: $\tau : S \times A \longrightarrow S$
  - Takes in a (STATE, ACTION) and returns a different STATE.
  - Defines the state the environment will transition to give the state and the action by the agent.
  - This transition function is deterministic as well
- A cost function: $c : S \times A \times S \longrightarrow \mathbb{R}_+$
  - Defines the reward received by the agent for a given transition
  - (STATE BEFORE, ACTION, STATE AFTER) $\longrightarrow$ COST
  - The input is called a transition. It says that I transitioned from one state to the other.
  - Notice how the cost also depends on the action.
  - You could replace this cost function with a reward function, but when we make the simplifying assumptions, we use a cost function instead.
  - The negative of the cost function might as well be a reward function.
- A set of goal states, $G \subseteq S$
  - States that end the process once reached

That's the setup to the search problem.

True RL can have multiple agents. We'll just assume that we only have one agent. Once we start relaxing assumptions we made, that's when we arrive to RL.

What is the agent going to do?

Assumptions:

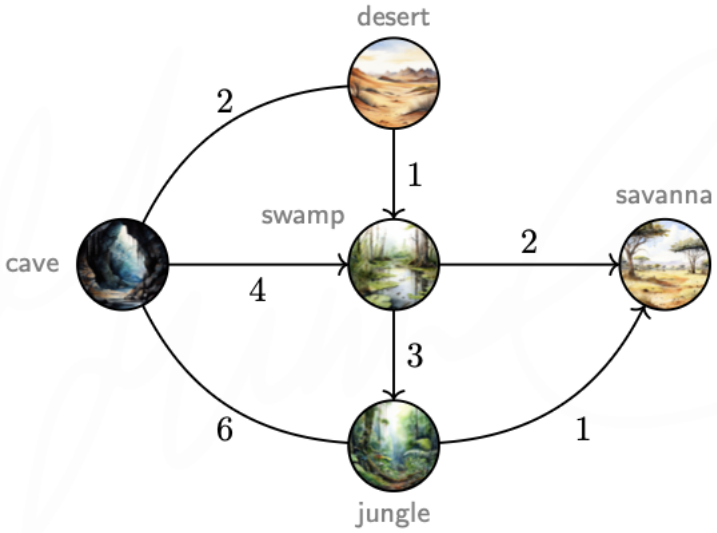
- Environment starts at initial state, $S_0 = s_0$
- $T$ is the max. no of transitions that could occur until the agent fails. It can be $\infty$.
- At each time step $t = 0, \ldots, T - 1$ the agent will:

  - **OBSERVE** the current state $S_t$
  - **SELECT** an action $A_t = \alpha(S_t)$
  - **MOVE** – act on the state $(S_t, A_t)$
- The environment evolves to a new state: $S_{t+1} = \tau(S_t + A_t)$
- The agent incurs a cost $C_t = c(S_t, A_t, S_{t+1})$
- Goals:
  - Find an optimal policy $\alpha^*$ to minimize the loss (total cost) after $S_T \in G$
  - $\sum_{t=0}^{T} C_t$

How can we visualize this?

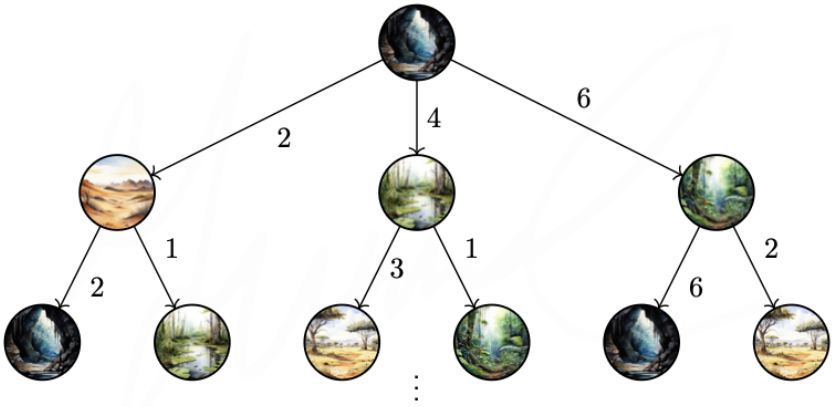
- Let's draw a state machine.
- Each arrow has a cost function.
- A transaction is a STATE-ACTION-STATE collection.
- A goal is the subset of the state.

Starts at cave, must go to savanna.



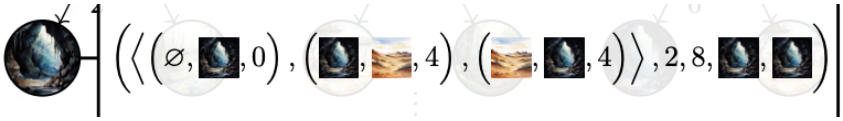

So, this is a shortest path problem. When you have a single agent and have deterministic environment, RL just becomes search. This is how AI started at.

This amounts to a tree of possible paths.




How deep is this tree? **Infinite.** This is the first problem with AI, where your algorithm might not work. (Branching factor: number of edges). No real-world AI problem will have a branching factor of 3. Your trees are incredibly large, and you won't have time to search for it.

**A node represents the entire path it took to get to that state.**



In the angle brackets, this encompasses the path. The path is a sequence of transitions. The first transition is a dummy transition (this ensures that transitions are consistent). At least you need to be able to store a path, or at least a reference to the path. But otherwise a vertex can be treated as a struct.

Our search algorithm wants to:

- Find a node where its path ends up in the goal state
- The cost is minimal

So our search algorithm:

1. Explore an open path
    a. Choose one of the open paths to explore. If none, stop and return None.

2. Check for a goal
3. Expand (continue to traverse) or export the chosen path

If our tree is very deep, how are we going to know if a solution exists or not? These are the problems we don't address in an algorithms class, but we do not really talk about when a tree is extremely huge.

## 2.1 The Search Algorithm

The search algorithm is as follows:

```
def search(graph: Graph,
           goals: dict[Key, Vertex],
           open: Queue[Path]) -> Optional[Path]:

    def _search(graph: Graph,
                goals: dict[Key, Vertex],
                open: Queue[Path]) -> Optional[Path]:
        if len(open) == 0:
            return None
        path = open.pop()
        if goals.get(path.dst.key) is not None:
            return path
        for edge in graph.get_edges_from(
                                    path.dst.key):
            new_path = path.expand(edge)  # copy
            open.push(len(new_path), new_path)
        return _search(graph, goals, open)

    open = Queue()
    dummy = Path()
    dummy.dst = start
    open.push(0, dummy)

    return _search(graph, goals, open)
```

Here's how the algorithm works, as posted above:

- I start with a dummy path (which is empty and the destination is the starting

vertex).

- I make a queue and add the dummy path into the queue.
- I retrieve the front of the queue if there is something there.
  - If the path's destination is a goal state, I have a working path. I return it.
  - Otherwise, for each edge from the current path's destination, I make a new path that extends the current one using that edge. I push them in the queue.
  - I repeat the process.

This is the BFS version of the search algorithm which does not do cycle checking. To make this the DFS version, replace the queue with the stack. Or really, you can make this strategy-agnostic by treating the queue as an abstract class that allows insertion and popping.

## 2.2 Properties of our Algorithm

We want our algorithm to be:

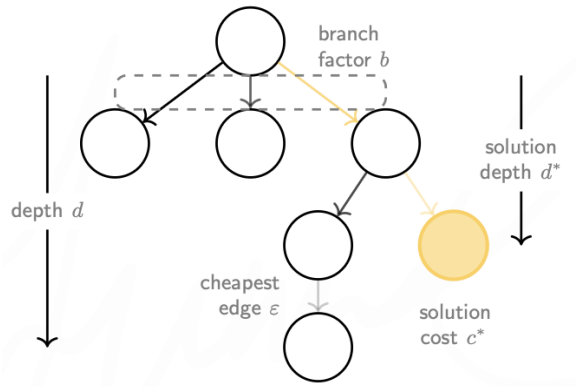
- Halting
  - Always returns a (possibly null) problem)
- Complete
  - Returns a non-null when a non-null solution exists (if there's a solution, it will say it's not null)
- Sound
  - Returns null when no solution exists
- Optimal
  - Returns the best solution when multiple non-null solutions exist
- Time-efficient

  - Minimal time

- Space-efficient
  - Minimal memory

We can't achieve all these goals at once, but there are tricks to get us as close as possible.

The tree that we have, will have some properties.




Note: you may see an off-by-one in the depth.

- $b$: The branch factor is the max no. of edge that leaves any node. In the image, there is no node that has more than 3 edges that leave it.
- $d$: The depth is 3 (if basing it on the edge count).
- $\varepsilon$: The weight of the **cheapest edge on the tree** (the smallest edge) will be denoted $\varepsilon$ here.
- $c^*$: The cost of the cheapest solution will be $c^*$.
- $d^*$: The depth of shallowest solution (the solution that is the shallowest, **ignoring the cost**) is $d^*$

So, let's study some of our algorithms:

| Property | BFS | DFS | CFS (cheap) |
|---|---|---|---|
| | Remove paths in increasing order of length | Remove paths in decreasing order of depth | Remove paths in increasing order of total cost |
| Halting | Y | | Y |
| Complete | Y | | Y |
| Optimal | | | Y |
| Time | $\mathcal{O}\left(b^d\right)$ | $\mathcal{O}\left(b^d\right)$ | $\mathcal{O}\left(b^{\lceil c^*/\varepsilon \rceil}\right)$ |
| Space | $\mathcal{O}\left(b^{d+1}\right)$ | $\mathcal{O}(bd)$ | $\mathcal{O}\left(b^{\lceil c^*/\varepsilon \rceil}\right)$ |
| Assume | $b < \infty$ | $b < \infty$ | $b < \infty$<br>$d < \infty$ |

The time and space complexity of an algorithm depends on the assumptions that are being made. If you are using $V$ and $E$, you will get a different time complexity. Here, we are expressing it with the branching factor ($b$) and the depth ($d$). We can't use $V$ and $E$ since these values might be infinite. So, think about complexity about the depth of the tree and the branching factor.

This lets us know some bad news: runtime is exponential.

Space is the maximum number of nodes that can be on the open set (also known as the frontier). That is the main difference between time and space complexity.

## 2.3 Cheapest-First Search (UCS)

CFS is optimal. (Isn't it Dijkstra's algorithm? It's simpler, though.)

The open set will be a priority queue.

*Remember: we don't stop searching until the path that reaches the goal leaves the open set. We may find out a cheaper path before we pull out the one we just added.*

The main point of CFS is because it is optimal, provided it is implemented correctly. There is a big con. The time and space complexity is terrible (exponential). Many AI problems have an infinite tree depth, and while the branching factor is usually not infinite, it is large.

So even if you see the exponential runtime, is it *that* bad? Many state-of-the-art AI uses CFS, but to only do part of the search. They use it for as long as they can, and when time runs out, something different will be done.

## 2.4 Cycle Checking

Wherever possible, it'll be great to make the algorithm faster. One of the most obvious things to do, assuming edge weights ar**e non-negative**, we ignore cyclic paths. (Non-negative weight cycles can cause an infinite-looping graph) This ensures that cycles are never added to a path.

The types of cycle checking are:

### 2.4.1 Intra-Path (Path checking)

Prevents revisiting vertices within the same path.

Paths are pruned (not explored) if duplicate vertices are discovered when traversing.

Before a node (path) is added to the open set, check it and don't add it if there's a cycle.

The idea is, paths are being checked for cycles.

### 2.4.2 Inter-Path (Cycle Checking)

Prevents revisiting vertices across different paths.

If a search leads to a certain state $A$, prune if another path traverses to any node that is state $A$.

When a goal is reached, add that state to the closed set (visited set). Auto-prune if that state is reached.

This type of pruning may break optimality. So why use this? It's faster.

### 2.4.3 When do Cycle Checks happen?

- Import time
  - Check before added to open set
- Export time
  - Check after added to open set

### 2.4.4 Iterative Deepening

Something used to improve memory.

Impose a depth limit. Prevents the algorithm from going further.

The depth limit starts at 0. Search algorithm is run on the tree. If solution, then good. If not, then increase depth limit by 1.

The time does not seem to be helped. But this improves memory. This also doesn't make time that much worse.

### 2.4.5 Iterative Inflating

Search to a fixed cost

Increase the cost per iteration

For the cost limit algorithm, it will be $\varepsilon$ to start off, the cost of the cheapest edge.

You can add $\varepsilon$, or you can add the cost of the cheapest path.

But maybe instead of improving memory, improve time.

# 3 Heuristics

Remember expert learning?

We try to capture existing knowledge to guide the algorithm to smarter decisions.

A caveman will still need to get to the savanna. Generally, the caveman has some intuition on where to go. Maybe going to the arctic is a bad idea. That's not a guarantee that the caveman will be right, but generally speaking, deserts are usually closer to the savanna than the swamp, since similar biomes tend to clump together.

So, the algorithm would go: try exploring the desert rather than the swamp. It's like how the grocery store stores things by category. Is there a guarantee that there won't be a shirt in the food section? No.

We want to encapsulate that information. So, what makes a path more promising?

## 3.1 Creating a Heuristic

For each state $s$, we can maintain an estimated cost to $\mathcal{G}$ using a function $h : S \to \mathbb{R}_+$.

We then define an estimated total cost function, $\widetilde{c}$, as follows:

Estimated total cost = current cost + estimated future cost (where `dst` is "destination")

- $\widetilde{c}(p)$: Estimated total cost from $s_0$ to $\mathcal{G}$ along $p$
- $c(p)$: Current cost from $s_0$ to $\text{dst}(p)$ along $p$
- $h(p)$: Estimated future cost from $\text{dst}(p)$ to $\mathcal{G}$
  - (in a slight abuse of notation, we use $h(p)$ to mean $h(\text{dst}(p))$).

$$\widetilde{c}(p) = c(p) + h(p)$$

The heuristic is specific to the goal.

The perfect heuristic tells you the actual cost to the goal. (a lower heuristic value means cheaper).

- $h^*(p)$: the ideal heuristic, actual cost to dst($p$) from $p$
- An over-estimate: $h(p) > h^*(p)$. You think it costs more than it actually does.
- An under-estimate: $h(p) < h^*(p)$. You think it costs less than it actually does.

Claim: If you had to decide between over-estimating and underestimating, **it will always be better to underestimate.** The reason, is we can show that so long as we have a heuristic that underestimates the cost, you will still have an optimal algorithm. If the heuristic overestimates, you may break optimality.

How are we going to use these heuristics?

## 3.2 Proof of why to underestimate

There are two properties of heuristics we want to look at:

### 3.2.1 Admissible

*Notation (recap):* $h^*(s)$ *is the actual cost to* dst($s$) *(the goal) from* $s$

**Definition** (Admissible)**.** $h(\cdot)$ is **admissible** if it always underestimates: $h(s) \leq h^*(s)$.

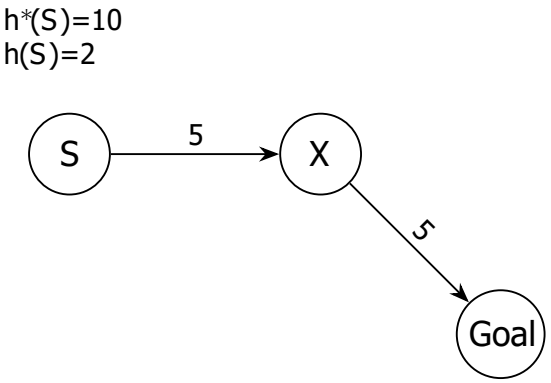



**Figure 1:** The heuristic h is admissible since it's less to the actual cost to the goal.

Admissible heuristics never overestimate the overall cost to reach the goal but may still overestimate transition costs.

**Consequence:** If we have an **admissible** heuristic, our $A^*$ algorithm remains optimal.

### 3.2.2 Consistent

*Notation:* $\text{tr}(s, a)$ *is the node* $\in S$ *that is attained after applying action* $a$ *when we are at* $s$.

**Definition** (Consistent)**.** A heuristic $h(\cdot)$ is **consistent** if for all $s \in S, a \in A(s)$:

$$\underbrace{h(s) - h(\text{tr}(s,a))}_{\text{estimated cost of transition } (s,a,\text{tr}(s,a))} \leq \underbrace{c(s,a,\text{tr}(s,a))}_{\text{true cost of transition } (s,a,\text{tr}(s,a))}$$

$$\text{and } h(s) = 0 \text{ for } s \in G$$

We could also rewrite this by moving something across the inequality:

$$h(s) \leq c(s,a,\text{tr}(s,a)) + h(\text{tr}(s,a))$$

A figure is attached:

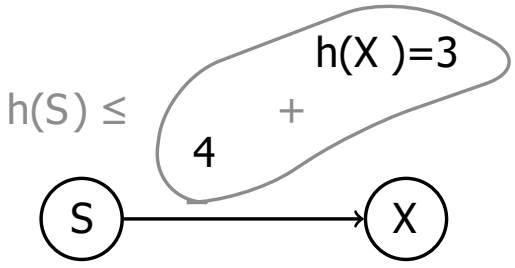



**Figure 2:** h(s) is consistent since it's less than the sum of the cost to transition to the next node, and the heuristic of that next node (X).

If you never overestimate the cost of an individual edge, you certainly won't overestimate the cost of the entire path. Consistent heuristic never over-estimate transition cost.

A heuristic is not admissible if at the goal state, the cost is anything other than 0.

Example:

$$\overset{(6)}{A} \overset{3}{\rightarrow} \overset{(2)}{B}$$

This heuristic predicts that the cost from $A$ to $B$ is $6 - 2 = 4$. It's not consistent because $4 > 3$.

Consistency $\Rightarrow$ Admissable

### 3.2.3 Strongly Dominate, Weakly Dominate

If $h_1$ and $h_2$ are admissible, then:

- Define $h_1$ to strongly dominate $h_2$ if for all $s \in S \setminus G$ (all states excluding the goal state, since they both need to be 0):
  - $h_1(s) > h_2(s)$
  - Strictly greater than $h_2$ everywhere
- Define $h_1$ to weakly dominate $h_2$ if for all $s \in S$:
  - $h_1(s) \geq h_2(s)$ and for *some* $s \in S$, $h_1(s) > h_2(s)$
  - $\geq$ to $h_2$ everywhere and $>$ $h_2$ somewhere

Heuristics that dominate are admissible but closer to the actual cost.

So, the statements to prove:

- Consistent $\Rightarrow$ Admissable
- If $\{h_k(\cdot)\}_k$ are admissible (resp. consistent) then $\max_k \{h_k\}(\cdot)$ is admissible (resp. consistent) and weakly dominates

(resp.: replace "admissible" with "consistent" and your statement remains true)

### 3.2.4 Consistency Implies Admissibility Full Proof

Recall: admissible means $h(s) \leq h^*(s)$ for all $s$. And consistent means: for all $s \in S, a \in A(s)$:

$$\underbrace{h(s) - h(\text{tr}(s,a))}_{\text{estimated cost of transition } (s,a,\text{tr}(s,a))} \leq \underbrace{c(s,a,\text{tr}(s,a))}_{\text{true cost of transition } (s,a,\text{tr}(s,a))}$$

$$\text{and } h(s) = 0 \text{ for } s \in G$$

WTS: If a heuristic $h$ is consistent, it's admissible.

Pick an arbitrary origin state $s_0$.

**Case 1:** If there is no path to $G$ from $s_0$, then $h(s_0)$ is infinite and the claim trivially holds (since no goal $\Rightarrow h^*(s) = \infty$) .

**Case 2:** Otherwise, let $p = \langle s_0, a_1, \ldots, a_n, s_n \rangle$ be an optimal path (note that this list stores vertex then edge then vertex…, where $a_n$ is the edge before $s_n$). We will prove **using reverse induction** that, for all $i = 0 \ldots n$, $h(s_i)$ is admissible.

**Base case:** Since $s_n \in \mathcal{G}$, then by definition of consistency, $h(s_n) = 0 = h^*(s_n)$. Thus, $h(s_n) \leq h^*(s_n)$. **This means the relationship of admissibility holds for the goal state,** $s_n$.

**Induction step:** Now, let's show that it holds, going backwards. But because I hate reverse induction, I'll use forward induction.

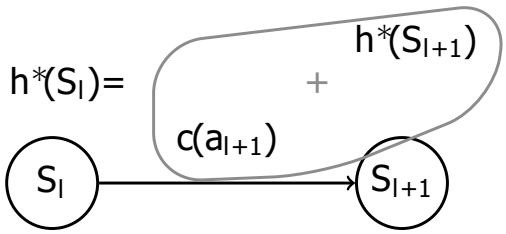
Assume $h(s_{l+1}) \leq h^*(s_{l+1})$ for some $l$.

Then, in order to prove $h(s_l) \leq h^*(s_l)$:

- $h(s_l) \leq c(a_{l+1}) + h(s_{l+1})$
  - Where $h^*(s_l) \rightarrow c(a_{l+1}) + h(s_{l+1})$
  - From the definition of consistency, I'm not overestimating the actual cost. Specifically, this fact: $h(s) \leq c(s,a,\text{tr}(s,a)) + h(\text{tr}(s,a))$
- $\leq c(a_{l+1}) + h^*(s_{l+1})$

  - Induction hypothesis: $h(s_{l+1}) \leq h^*(s_{l+1})$

- $\leq h^*(s_l)$
  - $c(a_{l+1}) + h^*(s_{l+1}) = h^*(s_l)$ (assuming path is simple). We use $\leq$ since we're upper-bounding – other paths could make $h^*(s_l)$ lower.
  - $(s_l) \underset{a_{l+1}}{\rightarrow} (s_{l+1})$, and you'll notice that $h^*(s_l)$ has to include everything to the right of $s_l$ (nodes themselves don't incur cost, it's just the edges).
  - Note to the attached figure: we use = since there are only two nodes. Normally, use $\leq$.




### 3.2.5 BFS Properties are as described – Proof

There are (at most) $b^l$ nodes on level $l$ of the tree. All nodes on level $l$ must be explored (and expanded) before any node on level $l+1$ is explored.

If the solution is on level $d$, then in the worst case, the algorithm will expand all (but not the solution) node on level $d$.

- On level $d^*$, there are $b^d$ nodes. I have to look at the children of at most $b^{d^*} - 1$ nodes, which means I'm performing expansions that many times.

So, we have: $b^0 + b^1 + \ldots + b^{d+1}$, which is after all, $\mathcal{O}\left(b^{d+1}\right)$.

But for the space complexity, I'm only concerned about max $\left\{b^0, \ldots, b^{d+1}\right\} = \mathcal{O}\left(b^{d+1}\right)$.

### 3.2.6 CFS (UCS) is Optimal

The properties of UCS is described: $\mathcal{O}\left(b^{\lceil c^*/\varepsilon \rceil}\right)$ in terms of time and space, assuming $b < \infty$, $d < \infty$.

I can prove the following result:

**We can show that the first time the algorithm explores a path $p$ to any state, it is the cheapest path to that destination.**

Proof by contradiction: assume that this isn't the case. Assume a cheaper path, $p'$ exists. Then either:

1. $p'$ was on the open set (frontier) with $p$ (I)
    a. Then $p'$ should've been explored before $p$ (inverse of II)
    b. At all times one of these have to be false, otherwise contradiction:
        i. $p'$and $p$ are on the open set at the same time
        ii. $p$ is explored (pulled out) before $p'$
        iii. $p'$ is strictly cheaper than $p$
    c. The combination of cases:
        i. I and II: $p$ being pulled out first $\Rightarrow$ anything pulled out after must be $\geq$ than the cost of $p$, which includes $p'$ since it's pulled out later (this is the contradiction)
        ii. I and III: then $p'$ has to be pulled out before $p$
        iii. II and III: the open set always pulls the cheapest item in the set. They couldn't have been together.
2. $p'$ is a successor (descendant of the tree) of $p$
    a. But successors must be more expensive than $p$ (edge weights are nonnegative)
3. $p'$ is a successor of another path $q$ that is on the frontier at the same time as $p$

a. Suppose the frontier = $\{p, q, \ldots\}$, where $p'$ is not there (since that's case 1). And case 2 prevents $p'$ from being on the frontier since $q$ must have been explored.

b. This means that any of $q$'s children must have a cost $\geq p$ (because I always add paths in a non-decreasing order into the frontier)

c. And therefore the cost of $p'$ is $\geq p$

In either case, we get a contradiction.

### 3.2.7 The A* Algorithm is Optimal

We show that the first time $A^*$ explores a path $p$ to a **goal node**, it is the cheapest path to the goal (if the heuristic is admissible).

Notice how this looks very similar to the UCS proof, but notice that I used the word "goal node" here.

Assume towards a contradiction that there is a cheapest path $p'$ to the goal. Then either:

1. $p'$ was on the frontier at the same time as $p$

   a. $\tilde{c}(p) = c(p')$ since $h$ is admissible and $h^*(p') = 0$ at the goal.

   b. Thus, $\tilde{c}(p') < c(p) = \tilde{c}(p)$, so $p'$ should have been explored before $p$.

   c. If you have an admissible heuristic, the estimated cost of $p'$ has to be less than the estimated cost of $p$. If that's the case, $A^*$ should have **pulled out** $p'$ first, under the presumption that $p'$ was on the open set at the same time.

      i. Recall: $\tilde{c}(p)$ is the actual + heuristic cost to the goal.

      ii. Note: we assume the heuristic is admissible.

   d. We state that $p$ is a path to a goal node. This means that $h^*(p)$ and $h^*(p')$ is 0.

2. $p'$ is a successor of $p$

   a. Impossible since $p$ ends at a goal

3. $p'$ is a successor of another path $q$ also on the frontier with $p$

   a. $\widetilde{c}(q) = c(q) + h(q) \leq c(q) + h^*(q) \leq c\left(p'\right)$ by admissiability. Thus $\widetilde{c}(q) < c(p) = \widetilde{c}(p)$ so that $q$ should've been explored before $p$

## 3.3 A* Algorithm and NFS

**NFS** (greedy best first search, nearest first search):

Use $\widetilde{c} = h$. Only use the heuristic. The estimated cost is just the heuristic value.

- Does not halt
- Not complete
- Not optimal
- If the heuristic is admissible, it's fast

**A$^*$ search**

Where we use $\widetilde{c}(p) = c(p) + h(p)$, where we incorporate the heuristic cost and the transition cost.

Notice that the time and space complexity remains the same, since our heuristic does not do much. However, it's:

- Halting
- Complete
- Optimal
- Time and space: $O\left(b^{\lceil c^*/\varepsilon \rceil}\right)$
- Assumes heuristics are admissible

## 3.4 Designing Heuristics / Problem Relaxation

If I have a problem and a solution with some cost, I take a relaxed version of that problem, I get a solution with some cost. That cost has to be $\leq$ the cost of the original problem.

In a relaxed problem, all I did is remove some of the restriction.

- Let $h^*_{\text{original}}$ be the perfect heuristic for a search problem, and $c^*_{\text{relaxed}}$ be the optimal cost for a relaxed version of the problem.
- Then $c^*_{\text{rel}}(s) \leq h^*_{\text{ori}}(s)$ for all $s \in S$. In other words, the relaxed problem underestimates the cost – it has fewer constraints.
- It follows that $c^*_{\text{rel}}$ can be used as an admissible heuristic for the original search problem.

But how do you relax a problem?

Let's say a caveman needs to cross a river from the near side to the far side. Let's make a map of it. Label *ground, forest, and river.*

You can model this as a search problem. Represent everything as a graph. Represent all the states the caveman could be in as a graph. Maybe as a grid, where every single cell is one possible state, and where the caveman initially was is $s_0$.

I assign edges to each of these nodes, and the weight will depend on the terrain type I am from and to. So now I know the cost of going from place to place. There can be multiple goals. You only need to get to one goal.

We can relax this problem and make it easier by simplifying the terrain by making all the terrain the same, smooth ground that costs barely anything to walk. This is **a** relaxed problem. We can now use this as a heuristic, if we run through the relaxed problem from our point. I can run the relaxed problem a bunch of times because it probably won't cost a lot. But at least, the heuristic is admissible because you reduced the weights to the cheapest possible and you removed all the constraints.

If you find a cheapest solution on the original problem, it has to be the cheapest solution in the relaxed problem. But not the other way around.

Today, we use ML and reinforcement learning to learn heuristics. No, they won't make the above obsolete. These are the components we use to make an overall optimal solution. All of this needs to be stitched together to design something useful.

### 3.4.1 What makes a relaxation good?

You want a heuristic so good you can trace it without searching. You should be able to tell that the solution in the original problem will be at least as expensive as the heuristic.

Maybe a harder constraint? What if I make the hard-to-walk areas ruled out? This is a hard constraint. I might also want to limit the goal to just one node.

When designing a relaxation, you want to make sure that the solution in the relaxation will be no more expensive than the one in the original. If you can show that, then you can use the solution of the relaxation as a heuristic in the original problem.

Again, **cost of the solution** in the relaxed problem should never be more expensive than the **cost in the original problem.**

A solution in a relaxed problem, with a constraint removed, will remain the solution and it will never be more expensive.

For example:

- If I can't swim, an optimal solution would be not utilizing swimming.
- If I can swim, the optimal solution doesn't need to take advantage of me swimming.
  - If swimming is a dumb idea, I don't need to do it.
- You will never get a worse solution in a relaxation.

Or:

- Bishops can now cut through pieces (like knights). If I use this heuristic, I will get a solution that might not be a valid solution, but it will give me a direction to start searching it and hopefully give me a faster solution.

### 3.5 Notation

- UCS (uniform-cost-search) $\longleftrightarrow$ CFS (cheapest-first search)
- Frontier $\longleftrightarrow$ open set
- Path-checking $\longleftrightarrow$ intra-path cycle checking
- Cycle-checking $\longleftrightarrow$ inter-path cycle checking
- $f \longleftrightarrow \widetilde{c}$ (estimated cost)
- $g \longleftrightarrow c$ (true cost so far)
- $h \longleftrightarrow h$ (heuristic)

## 4 Constraint Satisfaction

The purposes of these units is to give 5 different problems to solve in AI. The first problem was the reinforcement learning problem. We didn't get very far. We only covered the case where you are a single agent in a deterministic world. But what if you were a single agent in a stochastic world?

This is a different problem. This is a common problem AI has to solve. For CSP, we continue with this example:

## 4.1 CSP Examples

### 4.1.1 Caveman

Before hunts for food, he must make a weekly meal plan that satisfies his nutritional requirements.

His environment consists of various foods:


The meal plan must satisfy the following requirements:

- Variety
  - Each food must be eaten exactly once during the week
- Digestion
  - After eating mammoth or fish, next meal must be berries or potatoes
- Spoilage
  - Fish and mammoth must be eaten within the first three days
- Tribal
  - Eggs can only be eaten on certain days (1, 3, or 5)

There can be more than one way to model a CSP. Consider this constraint:

- Eggs can only be eaten on days 1, 3, and 5

We can model this in two ways:

- As a domain: egg $\notin$ dom $(V_2)$, egg $\notin (V_4)$
- As a constraint: $V_2 \neq$ egg, $V_4 \neq$ egg

The way the CSP is modelled can make it significantly easier or harder to solve.

Here's one possible model for our problem: One for each day, for days 1-5

$$V_1, V_2, V_3, V_4, V_5$$

Domains: each variable can be assigned any of the 5 foods (if we model it as a domain restriction):

$$\begin{aligned} &\text{All} \setminus \{\text{egg}\} \text{ for even days} \\ &\text{All otherwise} \end{aligned}$$

And then, our constraints. Well I'll just steal from the slides

- **constraints**
  the nutritional and preparation requirements

  variety
  each food must be eaten exactly once during the week

  $$\text{alldif}(V_1, \ldots, V_5)$$

  digestion
  after eating mammoth or fish, the next meal must be berries or potatoes

  $$V_i \in \{\text{🦣}, \text{🐟}\} \Rightarrow V_{i+1} \in \{\text{🫐}, \text{🍊}\}, i = 1, \ldots, 4$$

  spoilage
  fish and mammoth must be eaten within the first three days

  $$\text{🐟}, \text{🦣} \in \{V_1, V_2, V_3\}$$

  tribal
  eggs can only be eaten on certain days (1, 3, or 5)

  *accounted for with $\mathcal{D}_2$ and $\mathcal{D}_4$*

Do we have smaller constraints or less larger constraints? It depends on algorithm.

One possible solution for this CSP: $[\text{fish}, \text{berries}, \text{mammoth}, \text{potato}, \text{egg}]$. Notice how quick you can check the solution. But finding it is difficult. Not even LLMs with the reasoning model can solve this and can falsely claim that there are no solution to the problem. Why *can't* ChatGPT solve this problem?

### 4.1.2 Sudoku

Consider a 9x9 grid.

- One variable for each square, for a total of 81.
- **Domain:** The value for the pre-existing filled cells (e.g. a pre-existing cell that is 4 has the domain $\{4\}$), otherwise, $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for empty cells
- **State:** Any completed board given by specifying the value in each cell.
- **Partial state:** Some incomplete filling out of the board
- **Constraints:** The values assigned to the variables that form:
  - A column must be distinct
  - A row must be distinct
  - A sub-square must be distinct
- **Goal state:** A **state** that meets all the constraints.

For example, if the top left of a sudoku board is:

□ 2 □

□ □ □

□ 7 4

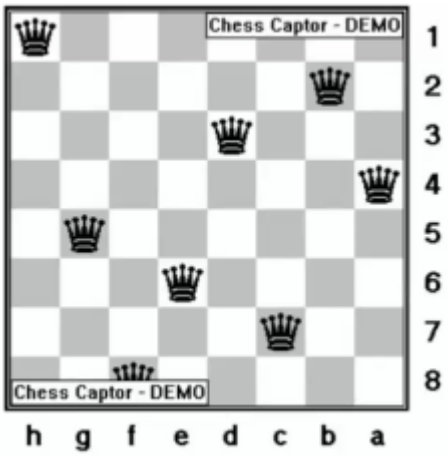
Then we would have these corresponding variables:

$$\begin{matrix} V_{11} & V_{12} & V_{13} \\ V_{21} & V_{22} & V_{23} \\ V_{31} & V_{32} & V_{33} \end{matrix}$$

Where $\text{Dom}[V_{12}] = \{2\}$, $\text{Dom}[V_{11}] = \{1, \ldots, 9\}$

### 4.1.3 Chess

Place $N$ queens on an $N \times N$ chess board so that no Queen can attack another queen.




Problem formulation:

- $N$ variables ($N$ queens)
- $N^2$ total possible values for each queen (the coordinate space)
- Total configurations? $\left(N^2\right)^N$, which blows up way too quickly ($64^8$). Alright. What can we do better?

Better formula: embed a constraint into a domain. We can't have more than one queen in a row, so we can have this:

- $N$ variables $Q_i$, where $i \in [1, N]$, one per row.
- Domain for each variable: $\{1,\ 2,\ \ldots,\ N\}$
- Total configurations: $N^N = 8^8 \approx 1.7 \times 10^7$

So, key idea is to come up with the most efficient representation for your problem. How can you capture the column and diagonal constraints?

- Column constraint: All-Diff $(Q_1, \ldots, Q_N)$
- Diagonal constraints: $\text{abs}(Q_i - Q_j) \neq \text{abs}(i - j)\ \forall i, j \mid i \neq j$

The total state count can be $\binom{N}{2}$.

## 4.2 The Basics

These are the requirements. This is a problem called a constraint satisfaction problem (CSP). The formalization is:

- A finite set of $n$ variables $V = \{V_1, V_2, \ldots, V_n\}$
- A domain $D_i$ for each variable $V_i$ (alt. notation: Dom $[V_i]$)
  - These are the possible values that can be assigned to this variable. The arity is the size of the domain. If there are 5 possible values, there is an arity of 5.
- A finite set of constraints, $C = \{C_1, C_2, \ldots, C_m\}$
  - Restrictions on which combinations of values are allowed
  - The constraint's scope is the variables it involves. For example, the scope of a constraint that involves $V_1, V_2, V_4$ is $\{V_1, V_2, V_4\}$.
  - The arity is the size of its scope
    - A binary constraint involves 2 variables
  - Constraints are True if and only if the assignment satisfies the constraint.
- A solution is a value assignment that meets all constraints.
- A CSP is unsatisfiable if no solution exists.

Unary constraints involve one variable; binary constraints involve two variables, higher-order constraints involve 3 or more. For example, All-Diff $(V_1, \ldots, V_n)$ means the values assigned to the $n$ variables must be distinct.

### 4.2.1 Definitions

An assignment is a mapping $\sigma : V \to \bigcup_i D_i$ (assigns a value from its domain to each variable). The codomain feels like using a union in Typescript – this could've been

done better using generics.

A constraint $C_j$ is satisfied if assignment meet its restriction (the constraint evaluates to true)

### 4.2.2 Goal

Find an assignment $\sigma*$ that meets the constraints

### 4.2.3 Representing CSPs

You can represent constraints by a table, draw every combination/permutation of each variable, and check if the constraints are met.

No, not very efficient, especially if the domain of the variables are large.

So instead, you use logic to represent the constraints. For example:

$$C(V_1, V_2, V_4) = (V_1 = V_2 + V_4)$$

## 4.3 The one thing LLMs can't replace

ChatGPT knows the type of the problem using its head. It admitted that it's a CSP. It knows the techniques. It lists out the algorithm. But it doesn't give us the right answer. What is it doing? Generally, these models are just string-language prediction. What they are really trained on is to figure out the next word that sounds natural. What it isn't doing, is applying logic. There are reasoning models that try to do that, but they reason by introspecting.

Logic is not over because LLMs exist.

Of course, the model isn't bringing out the Python module or writing code. Maybe, ask it to right down code that you could copy and run?

## 4.4 CSP Algorithms

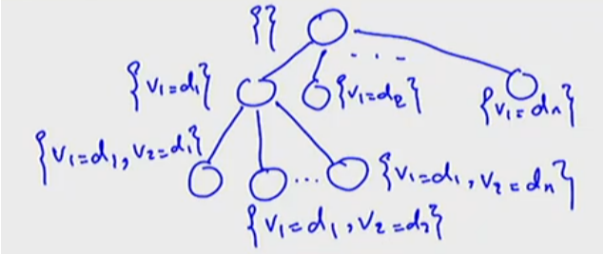
1. Check for solution
    a. If all variables are assigned, return the solution
2. Select variable
    a. Choose an unassigned variable $V_i$
3. Try value
    a. Choose a value $v \in D_i$ and assign $V_i = v$
4. Check consistency
    a. If assignment violates constraints, backtrack and try next value
5. Propagate
    a. Update domains of unassigned variables (algorithm specific)

The propagation step is the most interesting. After checking if an assignment meets a constraint, you should update the domains of the unassigned by pruning out variables that for sure can't be assigned.

Is this greedy? Yes, because this is a brute force algorithm.

Rather, a CSP could be formulated as a search problem:

- Initial state: Empty assignment
- Successor function: assign values to an unassigned variable.
- Goal test:
    - Assignment is complete.
    - No constraints violated.




CSPs do not require finding a path (to a goal). They only need the **configuration** of the goal state. CSPs are best solved by a specialized version of DFS called **Backtracking Search.**

Key intuitions:

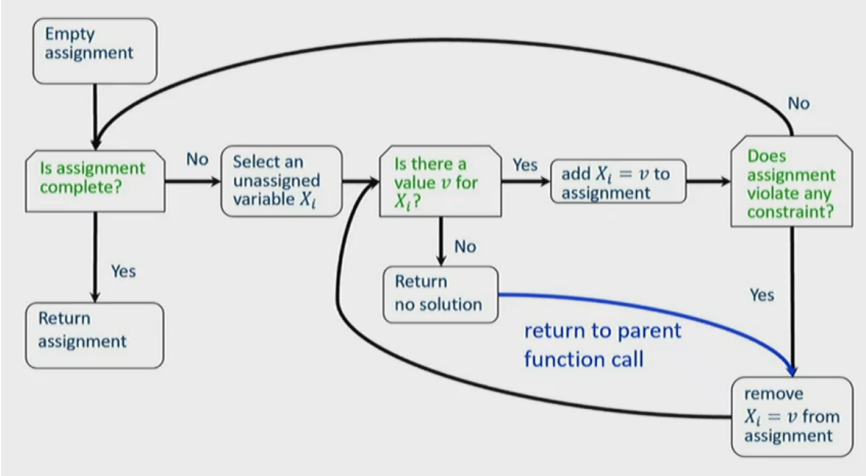
- Searching through the space of partial assignments, rather than paths.
- Decide on a suitable value for one variable at a time. Order in which we assign the variables do not matter.
- If a constraint is falsified during the process of partial assignment, immediately reject the current partial assignment.

## 4.5 Backtracking Search

We will apply recursive implementation:

- If all variables are set, print the solution and terminate.
- Otherwise:
  - Pick an unassigned variable *V* and assign it a value.
  - Test the constraints corresponding with *V* and all other variables of them are assigned.
  - If a constraint is unsatisfied, return (backtrack)
    - This is the base case
  - Otherwise, go one level deeper by invoking a recursive call.

```
def BT(Level, assignment):
1.  if all Variables assigned
2.     PRINT Value of each Variable
3.     EXIT or RETURN                  # EXIT for only one solution
                                       # RETURN for more solutions
4.  V := PickUnassignedVariable()
5.  Assigned[V] := TRUE
6.  for d := each member of Domain(V)   # the domain values of V
7.     add {V = d} to assignment
8.     ConstraintsOK := TRUE
9.     for each constraint C such that (i) V is a variable of C and
                                       (ii) all other variables of C are assigned:
10.         if C is not satisfied by the set of current assignments:
11.                ConstraintsOK := FALSE
12.                remove {V = d} to assignment
13.    if ConstraintsOk == TRUE:
14.         BT(Level+1, assignment)
15.  Assigned[V] := FALSE     # UNDO as we have tried all of V's values
16.  RETURN
```

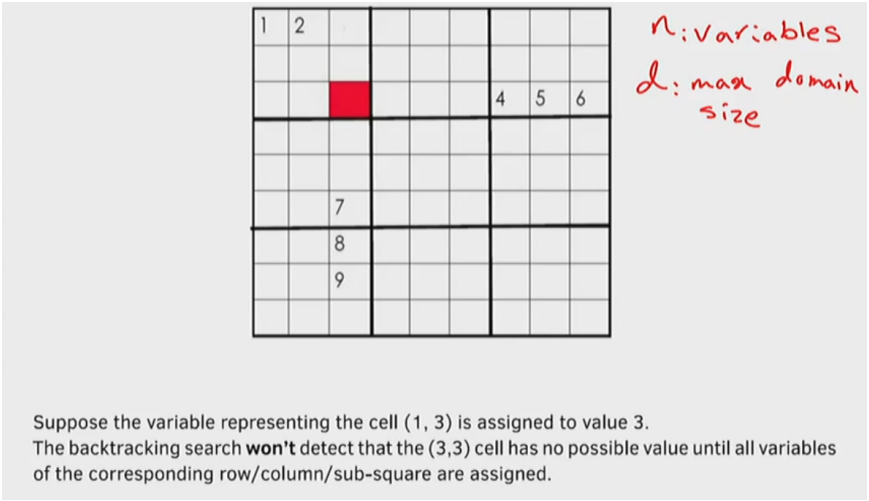



Example: the 4-queen problem

| Step | Assigning a Value and Revising Domains | | | | What Next? |
|---|---|---|---|---|---|
| | $Q_1$ | $Q_2$ | $Q_3$ | $Q4$ | |
| 1 | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | |
| 2 | $Q_1 = 1$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | Continue |
| 3 | | $Q_2 = 1$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | Constraint violated |
| 4 | | | | | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |
| 9 | | | | | |
| 10 | | | | | |
| | | | | | |

| Step | Assigning a Value and Revising Domains | | | | What Next? |
|---|---|---|---|---|---|
| | $Q_1$ | $Q_2$ | $Q_3$ | $Q4$ | |
| 1 | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | |
| 2 | $Q_1 = 1$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | Continue |
| 3 | | $Q_2 = 1$ | $1, 2, 3, 4$ | $1, 2, 3, 4$ | Constraint violated |
| 4 | | $Q_2=2$ | | | // |
| 5 | | $Q_2=3$ | | | Continue |
| 6 | | | $Q_3=1$ | | Violated |
| 7 | | | $Q_3=2$ | | // |
| 8 | | | $Q_3=3$ | | // |
| 9 | | | $Q_3=4$ | | // |
| 10 | | $Q_2=4$ | | | |
| | | | | | |

## 4.6 Problem with Pain Backtracking




Where $n$ is the no. of variables and $d$ is the max domain size. $n$ will be the depth of the tree.

Each layer corresponds to a variable assigned. $d$, the max domain size, is effectively the branching factor of the tree. Knowing all of these, the complexity of plain backtracking:

- Space: $O(nd)$
- Time: $O(d^n)$, when there is no solution

Without considering all possible configurations, we are pretty good at identifying failures.

STOPPED: recording 40:00